

1727. Largest Submatrix With Rearrangements

Solved 

Medium

 Topics

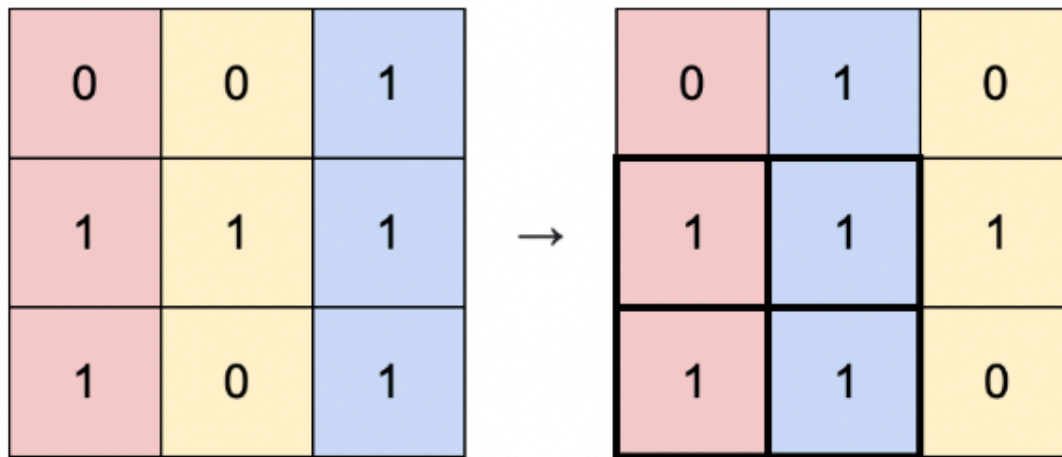
 Companies

 Hint

You are given a binary matrix `matrix` of size $m \times n$, and you are allowed to rearrange the **columns** of the `matrix` in any order.

Return *the area of the largest submatrix within `matrix` where **every** element of the submatrix is 1* after reordering the columns optimally.

Example 1:



0	0	1
1	1	1
1	0	1

 →

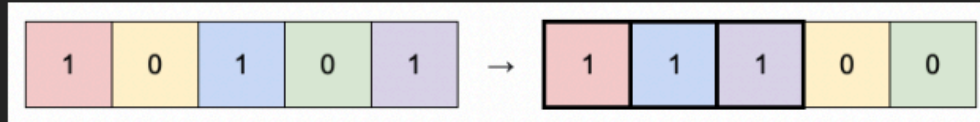
0	1	0
1	1	1
1	1	0

Input: `matrix = [[0,0,1],[1,1,1],[1,0,1]]`

Output: 4

Explanation: You can rearrange the columns as shown above. The largest submatrix of 1s, in bold, has an area of 4.

Example 2:



Input: `matrix = [[1,0,1,0,1]]`

Output: 3

Explanation: You can rearrange the columns as shown above. The largest submatrix of 1s, in bold, has an area of 3.

Example 3:

Input: `matrix = [[1,1,0],[1,0,1]]`

Output: 2

Explanation: Notice that you must rearrange entire columns, and there is no way to make a submatrix of 1s larger than an area of 2.

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m * n <= 105`
- `matrix[i][j]` is either 0 or 1.

Python:

class Solution:

def largestSubmatrix(self, matrix: List[List[int]]) -> int:

 m = len(matrix)

 n = len(matrix[0])

 res = 0

 for i in range(1, m):

 for j in range(n):

 if matrix[i][j] == 1:

 matrix[i][j] += matrix[i - 1][j]

 for i in range(m):

 matrix[i].sort(reverse=True)

 for j in range(n):

```
res = max(res, matrix[i][j] * (j + 1))
```

```
return res
```

JavaScript:

```
const largestSubmatrix = matrix => {  
  const m = matrix.length;  
  const n = matrix[0].length;  
  let max = 0;  
  
  for (let i = 1; i < m; i++)  
    for (let j = 0; j < n; j++)  
      if (matrix[i][j] === 1)  
        matrix[i][j] += matrix[i - 1][j];  
  
  for (let i = 0; i < m; i++) {  
    matrix[i].sort((j, k) => k - j);  
    for (let j = 0; j < n; j++)  
      max = Math.max(max, matrix[i][j] * (j + 1));  
  }  
  
  return max;  
};
```

Java:

```
class Solution {  
  public int largestSubmatrix(int[][] matrix) {  
    int m = matrix.length, n = matrix[0].length;  
    int res = 0;  
  
    for (int i = 1; i < m; i++)  
      for (int j = 0; j < n; j++)  
        if (matrix[i][j] == 1)  
          matrix[i][j] += matrix[i - 1][j];  
  
    for (int i = 0; i < m; i++) {  
      Arrays.sort(matrix[i]);  
      for (int j = 0; j < n; j++)  
        res = Math.max(res, matrix[i][j] * (n - j));  
    }  
  
    return res;  
  }  
}
```